Assignment – 6.5

2303A51860

Task 1 (AI-Based Code Completion for Conditional Eligibility Check)

PROMPT :

Generate Python code to check voting eligibility based on age and citizenship, Explanation of conditions.

CODE :

```python
#Generate Python code to check voting eligibility based on age and citizenship, Explanation of conditions.
# Python Program to Check Voting Eligibility Based on Age and Citizenship
def check_voting_eligibility(age, is_citizen):
    """
    This function checks if a person is eligible to vote based on their age and citizenship status.

    Parameters:
    age (int): The age of the person.
    is_citizen (bool): Citizenship status of the person (True if citizen, False otherwise).

    Returns:
    bool: True if eligible to vote, False otherwise.
    """
    # Condition 1: Age must be 18 or older
    # Condition 2: Must be a citizen
    if age >= 18 and is_citizen:
        return True
    else:
        return False
# Explanation of Conditions:
# 1. Age Condition: The legal voting age in many countries is 18 years. Therefore, the function checks if the provided age is 18 or older.
# 2. Citizenship Condition: Only citizens are typically allowed to vote in national elections. The function checks the boolean value of is_citizen to determine
# Example Usage
age = 20
is_citizen = True
print(f"Is the person eligible to vote? {check_voting_eligibility(age, is_citizen)}")
```

OUTPUT :

```
PS D:\AI>  & 'c:\Users\aatiq\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\aatiq\.vscode\ex
62796' '--' 'D:\AI\AI-6.5.py'
Is the person eligible to vote? True
PS D:\AI>
```

Justification :

This task highlights the practical use of conditional logic in validating user information. It helps understand how multiple conditions are combined to make accurate decisions. Such logic is widely applied in online forms, authentication systems, and eligibility verification platforms.

Task 2(AI-Based Code Completion for Loop-Based String Processing)

PROMPT :

Generate Python code to count vowels and consonants in a string using a loop.

CODE :

```python
#Generate Python code to count vowels and consonants in a string using a loop.
# Python Program to Count Vowels and Consonants in a String
def count_vowels_consonants(input_string):
    """
    This function counts the number of vowels and consonants in a given input string.

    Parameters:
    input_string (str): The string for which to count vowels and consonants.

    Returns:
    tuple: A tuple containing the count of vowels and consonants (vowel_count, consonant_count).
    """
    # Initialize counters for vowels and consonants
    vowel_count = 0
    consonant_count = 0

    # Define a set of vowels for easy checking
    vowels = set("aeiouAEIOU")

    # Loop through each character in the input string
    for char in input_string:
        # Check if the character is an alphabet letter
        if char.isalpha():
            # If it's a vowel, increment the vowel count
            if char in vowels:
                vowel_count += 1
            # If it's not a vowel, it must be a consonant, so increment the consonant count
            else:
                consonant_count += 1

    return vowel_count, consonant_count
# Example Usage
input_string = "Hello World!"
vowels, consonants = count_vowels_consonants(input_string)
print(f"Number of vowels: {vowels}")
print(f"Number of consonants: {consonants}")
```

OUTPUT :

```
PS D:\AI>  & 'c:\Users\aatiq\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\aatiq
56895' '--' 'D:\AI\AI-6.5.py'
Number of vowels: 3
Number of consonants: 7
PS D:\AI>
```

JUSTIFICATION :

This task enhances knowledge of loop-based string traversal. It demonstrates how individual characters are processed and classified, forming the foundation for text analysis, input processing, and language-based applications.

Task 3 (AI-Assisted Code Completion Reflection Task)

PROMPT :

Generate a Python program for a library management system using classes, loops, and conditional statements.

CODE :

```python
#Generate a Python program for a library management system using classes, loops, and conditional statements.
# Python Program for a Simple Library Management System
class Book:
    """
    A class to represent a book in the library.
    """
    def __init__(self, title, author):
        self.title = title
        self.author = author
        self.is_available = True  # Book is available by default

    def __str__(self):
        return f"'{self.title}' by {self.author} - {'Available' if self.is_available else 'Checked Out'}"
class Library:
    """
    A class to represent the library management system.
    """
    def __init__(self):
        self.books = []  # List to store books in the library

    def add_book(self, book):
        """
        Adds a book to the library.

        Parameters:
        book (Book): The book to be added.
        """
        self.books.append(book)
        print(f"Added: {book}")

    def display_books(self):
        """
        Displays all books in the library.
        """
        print("Library Books:")
        for book in self.books:
            print(book)

    def check_out_book(self, title):
        """
        Checks out a book from the library if it is available.

        Parameters:
        title (str): The title of the book to be checked out.
        """
        for book in self.books:
```

```python
    def check_out_book(self, title):
            if book.title == title:
                if book.is_available:
                    book.is_available = False
                    print(f"You have checked out: {book}")
                    return
                else:
                    print(f"Sorry, '{title}' is currently checked out.")
                    return
        print(f"Sorry, '{title}' is not found in the library.")

    def return_book(self, title):
        """
        Returns a book to the library.

        Parameters:
        title (str): The title of the book to be returned.
        """
        for book in self.books:
            if book.title == title:
                if not book.is_available:
                    book.is_available = True
                    print(f"You have returned: {book}")
                    return
                else:
                    print(f"'{title}' was not checked out.")
                    return
        print(f"Sorry, '{title}' is not found in the library.")
# Example Usage
library = Library()
book1 = Book("1984", "George Orwell")
book2 = Book("To Kill a Mockingbird", "Harper Lee")
library.add_book(book1)
library.add_book(book2)
library.display_books()
library.check_out_book("1984")
library.display_books()
library.return_book("1984")
library.display_books()
library.check_out_book("The Great Gatsby")
library.return_book("To Kill a Mockingbird")
library.check_out_book("To Kill a Mockingbird")
library.return_book("To Kill a Mockingbird")
library.register("user1", "Password123")
library.display_books()
library.login("user1", "Password123")
```

OUTPUT :

```
PS D:\AI>  & 'c:\Users\aatiq\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\aatiq\.vscode\extensions\ms-py
54770' '--' 'D:\AI\AI-6.5.py'
Added: '1984' by George Orwell - Available
Added: 'To Kill a Mockingbird' by Harper Lee - Available
Library Books:
'1984' by George Orwell - Available
'To Kill a Mockingbird' by Harper Lee - Available
You have checked out: '1984' by George Orwell - Checked Out
Library Books:
'1984' by George Orwell - Checked Out
'To Kill a Mockingbird' by Harper Lee - Available
You have returned: '1984' by George Orwell - Available
Library Books:
'1984' by George Orwell - Available
'To Kill a Mockingbird' by Harper Lee - Available
Sorry, 'The Great Gatsby' is not found in the library.
'To Kill a Mockingbird' was not checked out.
You have checked out: 'To Kill a Mockingbird' by Harper Lee - Checked Out
You have returned: 'To Kill a Mockingbird' by Harper Lee - Available
Library Books:
'1984' by George Orwell - Available
'To Kill a Mockingbird' by Harper Lee - Available
PS D:\AI>
```

JUSTIFICATION :

This task provides hands-on experience with object-oriented programming concepts. It shows how classes can organize data and behavior efficiently. This approach is essential for building scalable systems like library software, inventory management, and student databases.

Task 4 (AI-Assisted Code Completion for Class- Based Attendance System)

PROMPT :

Generate a Python class to mark and display student attendance using loops.

CODE :

```python
#Generate a Python class to mark and display student attendance using loops.
# Python Program to Mark and Display Student Attendance
class Student:
    """
    A class to represent a student and manage their attendance.
    """
    def __init__(self, name):
        self.name = name
        self.attendance_record = []  # List to store attendance status

    def mark_attendance(self, status):
        """
        Marks attendance for the student.

        Parameters:
        status (str): 'P' for present, 'A' for absent.
        """
        if status in ['P', 'A']:
            self.attendance_record.append(status)
            print(f"Attendance marked for {self.name}: {status}")
        else:
            print("Invalid status. Use 'P' for present and 'A' for absent.")

    def display_attendance(self):
        """
        Displays the attendance record of the student.
        """
        print(f"Attendance record for {self.name}:")
        for day, status in enumerate(self.attendance_record, start=1):
            print(f"Day {day}: {'Present' if status == 'P' else 'Absent'}")
# Example Usage
student = Student("Alice")
student.mark_attendance('P')
student.mark_attendance('A')
student.mark_attendance('P')
student.display_attendance()
```

OUTPUT :

```
PS D:\AI>  & 'c:\Users\aatiq\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\aatiq\.vscode\e
55303' '--' 'D:\AI\AI-6.5.py'
Attendance marked for Alice: P
Attendance marked for Alice: A
Attendance marked for Alice: P
Attendance record for Alice:
Day 1: Present
Day 2: Absent
Day 3: Present
PS D:\AI>
```

JUSTIFICATION :

This task focuses on structured data storage and retrieval using classes and loops. It represents real-world academic automation systems and helps develop skills in managing records programmatically.

Task 5 (AI-Based Code Completion for Conditional Menu Navigation)

PROMPT :

Generate a Python program using loops and conditionals

to simulate an ATM menu.

CODE :

```python
#Generate a Python program using loops and conditionals to simulate an ATM menu.
# Python Program to Simulate an ATM Menu
class ATM:
    """
    A class to simulate an ATM machine with basic functionalities.
    """
    def __init__(self, initial_balance=0):
        self.balance = initial_balance  # Initialize the account balance

    def display_menu(self):
        """
        Displays the ATM menu options.
        """
        print("\n--- ATM Menu ---")
        print("1. Check Balance")
        print("2. Deposit Money")
        print("3. Withdraw Money")
        print("4. Exit")

    def check_balance(self):
        """
        Displays the current account balance.
        """
        print(f"Your current balance is: ${self.balance:.2f}")

    def deposit_money(self, amount):
        """
        Deposits money into the account.

        Parameters:
        amount (float): The amount to deposit.
        """
        if amount > 0:
            self.balance += amount
            print(f"${amount:.2f} deposited successfully.")
        else:
            print("Deposit amount must be positive.")

    def withdraw_money(self, amount):
        """
        Withdraws money from the account if sufficient funds are available.

        Parameters:
        amount (float): The amount to withdraw.
        """
        if amount > 0:
            if amount <= self.balance:
```

```
                self.balance -= amount
                print(f"${amount:.2f} withdrawn successfully.")
            else:
                print("Insufficient funds for this withdrawal.")
        else:
            print("Withdrawal amount must be positive.")

    def run(self):
        """
        Runs the ATM simulation, displaying the menu and handling user input.
        """
        while True:
            self.display_menu()
            choice = input("Please select an option (1-4): ")
            if choice == '1':
                self.check_balance()
            elif choice == '2':
                amount = float(input("Enter amount to deposit: "))
                self.deposit_money(amount)
            elif choice == '3':
                amount = float(input("Enter amount to withdraw: "))
                self.withdraw_money(amount)
            elif choice == '4':
                print("Thank you for using the ATM. Goodbye!")
                break
            else:
                print("Invalid option. Please try again.")
# Example Usage
atm = ATM(1000)  # Initialize ATM with a starting balance of $1000
atm.run()
```

OUTPUT :

```
PS D:\AI>  & 'c:\Users\aatiq\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\aatiq\.vscode\extensions\ms
54865' '--' 'D:\AI\AI-6.5.py'

--- ATM Menu ---
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Please select an option (1-4): 1
Your current balance is: $1000.00

--- ATM Menu ---
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Please select an option (1-4): 2
Enter amount to deposit: 2000
$2000.00 deposited successfully.

--- ATM Menu ---
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Please select an option (1-4): 3
Enter amount to withdraw: 1000
$1000.00 withdrawn successfully.

--- ATM Menu ---
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Please select an option (1-4): 1
Your current balance is: $2000.00

--- ATM Menu ---
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Please select an option (1-4): 4
Thank you for using the ATM. Goodbye!
PS D:\AI>
```

JUSTIFICATION :

This task demonstrates interactive menu-driven programming. It teaches how user choices are handled dynamically using loops and conditionals, which is fundamental in financial applications, kiosks, and service-based systems.